

Particle Filters for Inference in Continuous State Space

Mohammed

September 2026

Abstract

The particle filter carries inference on a graphical model from discrete variables to continuous, non-Gaussian ones by representing each belief with weighted samples. This report treats one continuous problem under all three headings of a graphical model, representation, inference and learning. How well it works depends on its sampling choices: how new particles are proposed, how and when they are resampled, and how many there are. This report writes robot localization and simultaneous localization and mapping (SLAM) as dynamic Bayesian networks, derives the particle filter and FastSLAM, the Rao-Blackwellized particle filter that samples the robot's trajectory and estimates each landmark analytically, as approximate inference on them, and compares these choices in controlled experiments over 20 simulated worlds each. The four common resampling schemes give the same accuracy in localization, while systematic resampling keeps the most distinct particles; resampling adaptively, when the effective sample size or the largest weight signals degeneracy, is as accurate as resampling at every step, with low thresholds resampling at only a third to a half of the steps, and never resampling triples the error. Measurement-informed proposals keep a larger effective sample size: the auxiliary particle filter is the most accurate localization filter from 100 particles on, and FastSLAM 2.0 gives a better map than FastSLAM 1.0 with 5 to 10 particles. Beyond about 500 particles the error of the bootstrap filter improves only slowly while the runtime keeps growing. Finally the noise parameters of the network are estimated from the recorded runs by the marginal likelihood the filter's own weights provide: the two measurement parameters are recovered to within one step of the sweep and independent recordings agree on them, while the two motion parameters are undetermined by a 30-step run and identified on a 400-step one. Learning them needs the filter to track and enough particles for the likelihood estimate to be usable.

1 Introduction

Exact inference in probabilistic graphical models is usually developed for discrete variables. Variable elimination and belief propagation pass tables of numbers between factors, and summing out a variable is a sum over its values [9]. Many estimation problems in robotics do not fit this setting. The pose of a robot and the positions of landmarks are continuous, the motion and sensor models are nonlinear, and the belief about the pose can have several modes, for example when the robot does not yet know where it started. Summation becomes integration, and for these models the integrals have no closed form.

The particle filter handles this case by representing each belief with a set of weighted samples [7, 1]. It is approximate inference on a dynamic Bayesian network: the exact filtering recursion is kept, and each integral is replaced by importance sampling. The approximation is only as good as the sampling. A filter proposes new particles, weights them, and resamples them, and each of these steps has alternatives that trade accuracy against particle diversity and runtime. Which alternatives matter, and by how much, is the question of this report.

This report is organized around the three questions a graphical model raises, and its aim is to answer all three for one continuous problem. *Representation*: localization and SLAM are written as a dynamic Bayesian network, and the conditional independence its structure

implies, the landmarks being independent of one another given the trajectory, is exactly what makes Rao-Blackwellized SLAM tractable (Section 2). *Inference*: the particle filter is derived as approximate inference on that network, and the sampling choices it leaves open are compared experimentally (Sections 3 and 5). *Learning*: the parameters of the network’s two conditional distributions, the motion and the measurement noise, are estimated from a recorded run by maximum likelihood with the trajectory unobserved (Sections 2.4 and 5.5). Taking the three together is what the graphical-model view contributes here: the structure decides what can be factorized, the inference decides what can be computed, and the quality of the inference decides what can be learned.

The study uses two problems. In localization the map is known and the filter estimates the robot’s pose from a uniform start. In simultaneous localization and mapping (SLAM) the filter estimates the pose and a map of landmarks together; FastSLAM [10] does this with Rao-Blackwellization, sampling the robot’s trajectory and estimating each landmark analytically given that trajectory. On these two problems the report compares:

- the resampling scheme: multinomial, residual, stratified and systematic resampling;
- when to resample: at every step, never, or when the effective sample size or the largest weight shows that the weights have degenerated;
- the proposal distribution: the motion model against proposals that use the current measurement (the auxiliary particle filter, the extended Kalman particle filter, and FastSLAM 2.0);
- the number of particles.

A last question turns the setting around. Every filter above is told how noisy its motion and its sensors are. Those numbers are parameters of the two conditional distributions of the network, and the same weights that drive the filter also estimate how probable the recorded measurements are under them, so they can be learned from a run instead of being set by hand. The report asks which of them a run actually determines, and what has to be true of the inference for any of them to be learned.

The filters are built on existing open-source implementations: the particle filter tutorial of Elfring et al. [5] and the FastSLAM code of PythonRobotics [13]. Their sampling steps were made interchangeable, and the reused code was checked function by function against the original. That check found errors that would have biased the comparisons, most importantly a FastSLAM 2.0 implementation that never samples from its proposal; the corrections are listed in Appendix A.

Section 2 writes localization and SLAM as dynamic Bayesian networks, explains Rao-Blackwellization, and states what learning the noise parameters of such a network means. Section 3 derives the particle filter and the sampling choices compared here. Section 4 describes the simulated worlds, the implementation and the metrics, Section 5 presents the results, Section 6 discusses them, and Section 7 concludes.

2 Localization and SLAM as Graphical Models

2.1 Dynamic Bayesian network

A robot moves in the plane with pose $\mathbf{x}_t = (x_t, y_t, \theta_t)$ at time t . Between two time steps it receives a control $\mathbf{u}_t$ (a commanded motion) and then an observation $\mathbf{z}_t$ (ranges and bearings to landmarks). The map $\mathbf{m} = (\mathbf{m}_1, \dots, \mathbf{m}_K)$ holds the positions of K landmarks and does not change. These variables form a dynamic Bayesian network [9, ch. 6 and 15]: each pose depends

on the previous pose and the control, and each observation depends on the current pose and the map (Fig. 1). The joint distribution factorizes accordingly:

$$p(\mathbf{x}_{0:T}, \mathbf{m}, \mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}) = p(\mathbf{x}_0) p(\mathbf{m}) \prod_{t=1}^T p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t) p(\mathbf{z}_t \mid \mathbf{x}_t, \mathbf{m}). \quad (1)$$

The motion model $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$ and the measurement model $p(\mathbf{z}_t \mid \mathbf{x}_t, \mathbf{m})$ are the two conditional probability distributions of the network, repeated at every time slice.

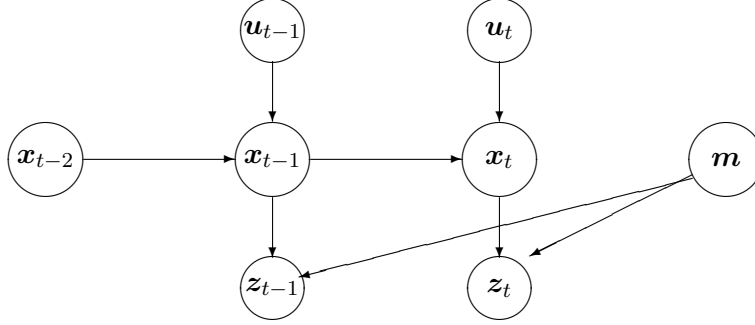


Figure 1: Two time slices of the dynamic Bayesian network for SLAM. Controls $\mathbf{u}$ and observations $\mathbf{z}$ are observed; poses $\mathbf{x}$ and the map $\mathbf{m}$ are hidden. In localization the map is known.

In *localization* the map is known and the task is to track the pose. Filtering means computing the belief $p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t})$ after each observation. Because $\mathbf{x}_t$ separates the past from the future in the network, the belief satisfies the recursion

$$p(\mathbf{x}_t \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \propto p(\mathbf{z}_t \mid \mathbf{x}_t) \int p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t) p(\mathbf{x}_{t-1} \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t-1}) d\mathbf{x}_{t-1}, \quad (2)$$

a prediction through the motion model followed by a correction with the likelihood of the new observation [14]. This is the forward pass of an unrolled network: sum-product message passing along the chain of poses.

In *SLAM* the map is unknown as well, and the target is the joint posterior $p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t})$. Throughout this report the correspondence between observations and landmarks is known: each observation carries the identity of the landmark it measures.

2.2 Rao-Blackwellization

The map has a structure that the network makes visible. Write the map as one node per landmark. An observation of landmark k at time s has exactly two parents, $\mathbf{x}_s$ and $\mathbf{m}_k$, because the correspondence is known. A path from $\mathbf{m}_j$ to $\mathbf{m}_k$ therefore has to leave $\mathbf{m}_j$ through one of its observations and reach that observation's pose $\mathbf{x}_s$, where both edges of the path point away from $\mathbf{x}_s$. If the whole trajectory $\mathbf{x}_{1:t}$ is given, $\mathbf{x}_s$ blocks the path. The observations on the path are observed colliders, but every such path continues through a given pose, so all paths are blocked: the landmarks are d-separated given the trajectory and the observations, and therefore conditionally independent [9]. The posterior factorizes as

$$p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) = p(\mathbf{x}_{1:t} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \prod_{k=1}^K p(\mathbf{m}_k \mid \mathbf{x}_{1:t}, \mathbf{z}_{1:t}). \quad (3)$$

The independence holds only given the full trajectory; given only the current pose, the landmarks are correlated through the uncertain path that observed them.

Rao-Blackwellization [4] uses this factorization to split the problem. The trajectory, the hard part with nonlinear and possibly multimodal uncertainty, is represented by samples. Given a

sampled trajectory each factor $p(\mathbf{m}_k \mid \mathbf{x}_{1:t}, \mathbf{z}_{1:t})$ is a small estimation problem over two variables, which FastSLAM solves with one extended Kalman filter per landmark [10]. Computing part of the posterior analytically instead of sampling it can only reduce the variance of the estimates: by the law of total variance, $\text{Var}[\mathbb{E}[f \mid \mathbf{x}_{1:t}]] \leq \text{Var}[f]$. It also keeps the cost low. Each particle holds K Gaussians of dimension two, instead of one Gaussian of dimension $2K + 3$, and an observation updates only the landmark it measures.

2.3 From discrete to continuous inference

For discrete variables the recursion (2) is the forward algorithm of a hidden Markov model: the belief is a table and the integral is a sum, computed exactly. For linear models with Gaussian noise the belief stays Gaussian and the recursion is the Kalman filter. Neither applies here. The measurement model, a range and a bearing to a landmark, is nonlinear, and after a uniform start the belief is spread over the whole world and has many modes. Linearizing, as the extended Kalman filter does, keeps a single mode and fails when the belief is far from Gaussian.

What carries over from the discrete case is the structure: the factorization (1), the conditional independences used by (2) and (3), and the message-passing view of filtering. What changes is how a message is represented. Sampling methods replace exact summation with sums over samples: likelihood weighting in a Bayesian network weights each sample by the likelihood of the evidence, which is importance sampling with the prior as the proposal [9, ch. 12]. The particle filter applies this idea at every time slice of the dynamic network and adds resampling so that the samples stay useful over time [9, ch. 15].

2.4 Learning the parameters

The two conditional distributions above are given, but each carries a noise level that somebody has to choose. Collect those numbers in θ : here four standard deviations, for the forward motion, the turn, the measured range and the measured bearing. Estimating them from data is *parameter* learning rather than structure learning, since the network of Fig. 1 is fixed and only the numbers inside two of its conditional distributions are unknown. It is parameter learning *with incomplete data*, because the trajectory the observations depend on is never observed [9, ch. 19]: the data are the controls and the observations of a recorded run, and nothing else.

A single θ covers the whole run because a dynamic network is a template model. The same conditional distribution is instantiated at every time slice, so its parameters are tied across slices [9, ch. 6]. Every one of the T steps is then evidence about the same four numbers, and a longer trajectory is more data about θ rather than more unknowns.

The maximum-likelihood estimate is the θ under which the recorded observations are most probable,

$$\hat{\theta} = \arg \max_{\theta} \log p(\mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}, \theta), \quad p(\mathbf{z}_{1:T} \mid \mathbf{u}_{1:T}, \theta) = \prod_{t=1}^T p(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}, \theta). \quad (4)$$

Each factor integrates over all trajectories and has no closed form, but the particle filter estimates it as a by-product. With $w_{t-1}^{(i)}$ the normalized weights after step $t - 1$ and particles propagated through the motion model, the unnormalized weight of particle i is $\tilde{w}_t^{(i)} = w_{t-1}^{(i)} p(\mathbf{z}_t \mid \mathbf{x}_t^{(i)}, \theta)$ and

$$p(\mathbf{z}_t \mid \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}, \theta) \approx \sum_{i=1}^N \tilde{w}_t^{(i)}, \quad (5)$$

which is the sum the filter forms anyway in order to normalize its weights [3]. Adding the logarithm of (5) over the run estimates $\log p(\mathbf{z}_{1:T} \mid \theta)$. The estimate of the likelihood is unbiased, so the estimate of its logarithm is biased downwards, and that bias grows as the particle

approximation degrades. The same machinery is therefore both the inference and the objective that learning maximizes, which ties the quality of one to the other.

Two properties of (4) decide whether anything can be learned from it. The first is that its maximum is in the right place. As the run grows the average log likelihood tends to an expectation that splits into a term free of θ and a Kullback-Leibler divergence,

$$\frac{1}{T} \log p(\mathbf{z}_{1:T} | \theta) \longrightarrow \mathbb{E}_{p^*}[\log p^*(\mathbf{z})] - \text{KL}(p^* \| p_\theta), \quad (6)$$

and the divergence is non-negative and zero only where the two distributions agree [9, ch. 17]. The expected score is therefore largest at the parameters that generated the data: maximizing the likelihood is minimizing the distance to the true model.

The second is how sharp that maximum is. Expanding the expected score around θ^* gives

$$\mathbb{E}[\log p(\mathbf{z}_{1:T} | \theta)] \approx \text{const} - \frac{1}{2} T \mathcal{I}(\theta^*) (\theta - \theta^*)^2, \quad (7)$$

with $\mathcal{I}$ the information a single step carries about θ , and the variance of the estimate is $1/(T\mathcal{I})$. A parameter the observations say nothing about has $\mathcal{I}$ near zero and a flat likelihood: it is part of the model and still not identifiable from the run. Because the parameters are tied across slices, the curvature grows with the number of steps, so a parameter that a short trajectory cannot resolve can be resolved by a longer one.

The shape on either side of the maximum follows from the same expression. For one Gaussian residual with true standard deviation σ^* , scored under an assumed σ ,

$$\mathbb{E}[\log p] = -\log \sigma - \frac{\sigma^{*2}}{2\sigma^2} - \frac{1}{2} \log 2\pi, \quad (8)$$

whose derivative $-1/\sigma + \sigma^{*2}/\sigma^3$ vanishes at $\sigma = \sigma^*$. The two terms are not symmetric: assuming too little noise is penalized like σ^{-2} , assuming too much only like $\log \sigma$. Understating a noise level is therefore punished far more heavily than overstating it, which is a reason to set such a value generously when it is set by hand.

Maximizing (4) by sweeping one component of θ at a time, with the others held fixed, is a profile likelihood. A general method would instead iterate, alternating a step that reweights trajectories under the current θ with a step that updates θ in closed form, which is expectation maximization [9, ch. 19] applied to a state-space model. The sweep is used here because it also shows the shape of the likelihood, and that shape is what decides whether a parameter can be identified at all.

3 Particle Filters as Approximate Inference

3.1 Sequential importance sampling

Importance sampling estimates expectations under a target density p from samples of a proposal density q that is easier to sample: with $\mathbf{x}^{(i)} \sim q$ and weights $w^{(i)} \propto p(\mathbf{x}^{(i)})/q(\mathbf{x}^{(i)})$ normalized to sum to one, $\mathbb{E}_p[f] \approx \sum_i w^{(i)} f(\mathbf{x}^{(i)})$. The particle filter applies this to the posterior over trajectories, $p(\mathbf{x}_{0:t} | \mathbf{z}_{1:t}, \mathbf{u}_{1:t})$, and builds the samples one time step at a time [1, 3]. Each of the N particles extends its trajectory with a new pose drawn from a proposal $q(\mathbf{x}_t | \mathbf{x}_{t-1}, \mathbf{z}_t, \mathbf{u}_t)$, and by the factorization (1) its weight is updated recursively:

$$w_t^{(i)} \propto w_{t-1}^{(i)} \frac{p(\mathbf{z}_t | \mathbf{x}_t^{(i)}) p(\mathbf{x}_t^{(i)} | \mathbf{x}_{t-1}^{(i)}, \mathbf{u}_t)}{q(\mathbf{x}_t^{(i)} | \mathbf{x}_{t-1}^{(i)}, \mathbf{z}_t, \mathbf{u}_t)}. \quad (9)$$

The weighted particles approximate the filtering belief (2), and the pose estimate is their weighted mean.

The variance of the weights can only grow over time [3]. After a few steps almost all the weight sits on a few particles and the rest contribute nothing, which is called weight degeneracy. A common measure is the effective sample size

$$\text{ESS} = \frac{1}{\sum_{i=1}^N (w_t^{(i)})^2}, \quad (10)$$

which is N for equal weights and 1 when one particle holds all the weight. Two things work against degeneracy: a proposal that places particles where the posterior is high, and resampling.

3.2 Proposal distributions

The *motion-model* (bootstrap) proposal $q = p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$ [7] cancels the motion term in (9), so the weight is just the likelihood of the observation. It is simple, but it ignores $\mathbf{z}_t$ when sampling: if the sensor is more precise than the motion, most particles land where the likelihood is small.

Given $\mathbf{x}_{t-1}$, the proposal that minimizes the variance of the weight is $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{z}_t, \mathbf{u}_t)$, the pose distribution after both the motion and the new observation [3]. It rarely has a closed form, and the measurement-informed proposals approximate it:

- The *extended Kalman particle filter* gives each particle an extended Kalman filter. The filter predicts the pose with the motion model, updates it with the linearized measurement model, and the new pose is drawn from the resulting Gaussian; the weight then follows (9) with that Gaussian as q [5].
- The *auxiliary particle filter* [12] first decides which particles to continue. It propagates each particle once, weights it by how well the propagated pose explains $\mathbf{z}_t$, and resamples by these first-stage weights. The selected particles are then propagated again with the motion model, and their weights are divided by the first-stage likelihood.
- *FastSLAM 2.0* [11] samples the pose from a Gaussian that combines the predicted pose with the observations of already mapped landmarks, including the uncertainty of those landmarks. The weight is the predictive likelihood of the observations. *FastSLAM 1.0* [10] uses the motion-model proposal.

Measurement-informed proposals cost more per particle. They should therefore reach a given accuracy with fewer particles; whether they pay off at a given runtime is an empirical question.

3.3 Resampling

Resampling draws N new particles from the current ones, with probability proportional to their weights, and resets all weights to $1/N$. Particles with negligible weight disappear and heavy particles are copied. The schemes differ in how much randomness they add to the number of copies N_i of particle i , whose expectation is $Nw^{(i)}$ in all four [2, 8]:

- *Multinomial*: N independent draws from the weights, so N_i is binomial.
- *Residual*: $\lfloor Nw^{(i)} \rfloor$ copies of each particle deterministically, and the remaining particles by multinomial sampling from the residual weights.
- *Stratified*: the interval $[0, 1)$ is split into N equal strata, one uniform number is drawn in each, and each number selects the particle whose cumulative weight covers it.
- *Systematic*: a single uniform number $u \in [0, 1/N)$ gives the N positions $u + (j - 1)/N$, which select particles in the same way.

Residual, stratified and systematic resampling have lower variance than multinomial resampling [2], and all four can be implemented in $O(N)$ time.

Resampling removes weight degeneracy but causes sample impoverishment: after resampling many particles are copies of the same pose, and the diversity lost cannot be recovered. Resampling at every step, as the original bootstrap filter does, therefore throws away information when the weights are still balanced. Adaptive rules resample only when the weights have degenerated. The rules compared here resample when $ESS < \tau N$, or when the largest weight is too large, $1/\max_i w^{(i)} < \tau N$, for a threshold $\tau \in (0, 1)$ [5].

3.4 MCMC moves

Resample-move algorithms [6] restore diversity after resampling by applying a Markov chain Monte Carlo kernel that leaves the posterior invariant, such as a few Metropolis-Hastings steps, to each particle. Copies of the same particle then move apart without changing the distribution they represent. The experiments in this report do not use moves; they are a natural next step (Section 6).

4 Method

4.1 Filters and worlds

Localization. The localization problem is the running example of the particle filter tutorial by Elfring et al. [5], simulated with its own classes. The world is a 10×10 m square with four landmarks at (2, 2), (2, 8), (9, 2) and (8, 9) m; positions wrap around at its edges. The robot starts at (7.5, 2) m facing along the y axis and makes 30 steps, each commanding a forward motion of 0.25 m and a turn of 0.02 rad. Its true motion has noise with standard deviations 0.005 m and 0.002 rad, and after each step it measures the range and bearing to all four landmarks with noise of 0.2 m and 0.05 rad. The filter assumes larger noise, 0.1 m and 0.2 rad in motion and 0.4 m and 0.3 rad in measurement, the settings of the tutorial. It does not know the start: the particles are drawn uniformly over the world and all headings, so the filter solves global localization during the first steps. The learning experiments of Section 5.5 also use a second setting, in which the particles start around the true initial pose with a standard deviation of 0.2 m and 0.2 rad, so that the filter tracks a robot whose starting point is roughly known.

SLAM. The SLAM problem is the FastSLAM scenario of PythonRobotics [13], simulated with its own functions. The robot waits for 3 s and then drives at 1 m/s with a yaw rate of 0.1 rad/s for 50 s in total, 500 steps of 0.1 s, among eight landmarks. It observes the range and bearing of every landmark within 20 m, with noise of 0.3 m and 2° , and the filter receives the control with noise of 0.5 m/s and 10° /s plus a constant yaw-rate offset of 0.01 rad/s. The filter assumes noise of 3 m and 10° for observations and 1 m/s and 20° /s for controls, the settings of the upstream script. All particles start at the true initial pose, and the landmark identities are known. Each particle carries one extended Kalman filter per landmark it has observed.

Seeds. Every configuration runs with 20 seeds. A seed fixes both the simulated world (the true path and all measurements) and the random numbers of the filter, and every variant of an experiment uses the same seeds, so differences between variants within a seed come from the sampling choice alone.

4.2 Implementation

The filters are built on two open-source implementations: the Python particle filter tutorial of Elfring et al. [5] for localization and the four resampling schemes, and the FastSLAM 1.0

and 2.0 scripts of PythonRobotics [13]. Both are MIT-licensed and pinned to fixed commits (Appendix A). The four resampling algorithms and the three rules for when to resample are called unchanged from the tutorial. The motion and measurement models, the proposals and the FastSLAM updates are copied, vectorized over particles and changed to compute weights in log space.

Each filter runs the same loop at every step: the proposal draws new poses and returns log weights, the weights are normalized, the effective sample size (10) is recorded, the rule decides whether to resample, and if so the resampling scheme selects the particles. Proposals, resampling schemes and rules are interchangeable components selected by name, so an experiment is a configuration file that lists the settings to keep fixed and the settings to vary. Every run is stored with its settings, metrics, per-step values and a fingerprint of the code that produced it.

Before any experiment, each copied function was compared with the original. With the corrections switched off, the copies reproduce the original results: the localization likelihood and motion model exactly, one full extended Kalman particle filter update, and the FastSLAM prediction and observation updates over several steps. The comparison also exposed errors in the original code, among them a FastSLAM 2.0 update that never samples from its proposal and an extended Kalman particle filter whose weights break at the world’s edges. The experiments use the corrected versions, described in Appendix A.

The experiments ran in a container with Python 3.11 and NumPy 2.2, with 16 runs in parallel on a 52-core computer. Runtimes are therefore comparable within an experiment but only indicative in absolute terms.

4.3 Metrics

Let $\hat{\mathbf{x}}_t$ be the weighted mean of the particles after each step (the heading as a circular mean) and $\mathbf{x}_t$ the true pose, over T steps.

- *Position RMSE*: $\sqrt{\frac{1}{T} \sum_t \|\hat{p}_t - p_t\|^2}$, with p the position part of the pose.
- *ATE* (absolute trajectory error): the same after the best rigid alignment (rotation and translation) of the estimated path onto the true one. In SLAM the map frame is only anchored at the start, so a rotation of the whole solution is not an error in its shape; the ATE removes it. In localization it is close to the RMSE.
- *Map error (aligned)*: the mean distance between the estimated and true landmark positions at the end of a SLAM run, after the best rigid alignment of the estimated map.
- *Mean ESS/ N* : the effective sample size before resampling, divided by N and averaged over steps.
- *Distinct particles after resampling/ N* : the number of different particles that survive a resampling step, divided by N and averaged over the steps that resample. It measures impoverishment.
- *Resampling rate*: the fraction of steps that resample.
- *Median NEES*: the normalized estimation error squared, $e_t^\top S_t^{-1} e_t$ with e_t the pose error and S_t the weighted covariance of the particles, whose median over the steps of a run is reported. For a filter whose spread matches its error the NEES is about 3, the pose dimension; much larger values mean the particles are overconfident. The median is used because the first steps after a uniform start give values of 10^8 and more that would dominate a mean.
- *Log likelihood*: $\sum_t \log \sum_i \tilde{w}_t^{(i)}$ over a run, the estimate (5) of $\log p(\mathbf{z}_{1:T} \mid \theta)$ under the noise settings the filter assumes. It is reported only for the motion-model proposal, whose

weights take the form (5) requires; the auxiliary and extended Kalman proposals weight in two stages and would need a different estimator.

- *Runtime per step*: wall-clock time of the filter step, without simulation and logging.

Each table reports the mean and standard deviation over the 20 seeds. A difference between two variants is called clear when their 95% confidence intervals for the mean, $\bar{m} \pm 1.96 s/\sqrt{20}$, do not overlap, and within the seed-to-seed spread otherwise. This test is conservative, and it is applied only between variants with the same value of the other varied setting, for example between resampling schemes at the same number of particles.

5 Experiments and Results

Each experiment varies one sampling choice, sometimes together with the number of particles, and keeps everything else fixed. The tables give the mean and standard deviation over 20 seeds; the figures show the mean with its 95% confidence interval. A difference is called clear when the 95% intervals do not overlap (Section 4).

5.1 Resampling scheme

Question and setup. Do the four resampling schemes give different accuracy or particle diversity when everything else is equal? The localization filter uses the motion-model proposal, resamples at every step so that only the scheme differs, and runs with 50, 200 and 1000 particles. The same comparison in FastSLAM 1.0 uses 10 and 50 particles. The expectation was that the lower-variance schemes keep more distinct particles and give slightly lower error than multinomial resampling, most visibly with few particles.

Results. In localization (Table 1) the schemes are equally accurate. At every particle count their position errors differ by less than the seed-to-seed spread; with 1000 particles all four reach 0.13 m, and at 200 particles 0.25 to 0.27 m. They differ clearly in diversity. After a resampling step, systematic resampling keeps 72% distinct particles with 1000 particles, stratified 67%, residual 65% and multinomial 54%, and the same order holds at 50 and 200 particles. The effective sample size before resampling does not depend on the scheme, since it is computed before the scheme acts.

In FastSLAM 1.0 (Table 2) the order of diversity is the same, from 86% distinct particles for systematic to 61–62% for multinomial resampling, and here it carries over to accuracy. With 10 particles systematic resampling gives a clearly lower trajectory error than multinomial resampling (ATE 0.23 against 0.32 m) and a clearly better map (0.105 against 0.139 m). With 50 particles stratified resampling gives the best map, clearly better than multinomial (0.069 against 0.097 m).

The runtimes of the four schemes reflect the reused implementations rather than the schemes themselves. The multinomial resampler searches the cumulative weights linearly for every new particle, and the stratified resampler recomputes the sum of all weights for every particle, so both take $O(N^2)$ time; the residual resampler passes its remaining particles to the same multinomial search. The systematic resampler makes one pass over the weights, $O(N)$. All four schemes can be implemented in $O(N)$, so only the accuracy and diversity results compare the schemes; the runtime column compares these implementations.

5.2 When to resample

Question and setup. Should the filter resample at every step, never, or only when the weights have degenerated, and how much does the threshold matter? The localization filter uses 100

Table 1: Resampling schemes in localization, resampling at every step. Mean $\pm$ standard deviation over seeds.

Resampler	Particles	Seeds	Position RMSE (m)	ATE (aligned) (m)	Distinct after resampling / N	Mean ESS / N	Median NEES	Runtime per step (ms)
Multinomial	50	20	0.784 ± 1	0.481 ± 0.42	0.472 ± 0.12	0.56 ± 0.19	$1.96e+08 \pm 8.7e+08$	0.554 ± 0.012
Multinomial	200	20	0.252 ± 0.22	0.196 ± 0.15	0.525 ± 0.035	0.668 ± 0.083	9.18 ± 37	2.16 ± 0.029
Multinomial	1000	20	0.129 ± 0.023	0.107 ± 0.018	0.541 ± 0.0055	0.705 ± 0.016	0.447 ± 0.16	30.8 ± 0.22
Residual	50	20	0.672 ± 0.58	0.465 ± 0.3	0.558 ± 0.12	0.555 ± 0.17	$651 \pm 1.9e+03$	0.695 ± 0.03
Residual	200	20	0.269 ± 0.2	0.21 ± 0.15	0.624 ± 0.046	0.656 ± 0.08	2.67 ± 7.4	2.19 ± 0.067
Residual	1000	20	0.13 ± 0.023	0.108 ± 0.018	0.65 ± 0.0099	0.705 ± 0.016	0.469 ± 0.19	19.9 ± 0.88
Stratified	50	20	0.851 ± 1.1	0.502 ± 0.42	0.566 ± 0.16	0.542 ± 0.21	$2.63e+05 \pm 1.2e+06$	0.633 ± 0.014
Stratified	200	20	0.247 ± 0.2	0.196 ± 0.14	0.652 ± 0.042	0.665 ± 0.074	3.72 ± 14	3.03 ± 0.017
Stratified	1000	20	0.13 ± 0.025	0.107 ± 0.02	0.673 ± 0.0093	0.706 ± 0.016	0.458 ± 0.17	49.1 ± 1.7
Systematic	50	20	0.649 ± 0.47	0.463 ± 0.28	0.619 ± 0.11	0.557 ± 0.16	$265 \pm 6.6e+02$	0.299 ± 0.01
Systematic	200	20	0.252 ± 0.18	0.196 ± 0.12	0.687 ± 0.052	0.659 ± 0.078	4.51 ± 17	0.592 ± 0.013
Systematic	1000	20	0.13 ± 0.025	0.107 ± 0.019	0.715 ± 0.011	0.705 ± 0.016	0.448 ± 0.18	2.11 ± 0.5

Does the resampling scheme matter? (localization)

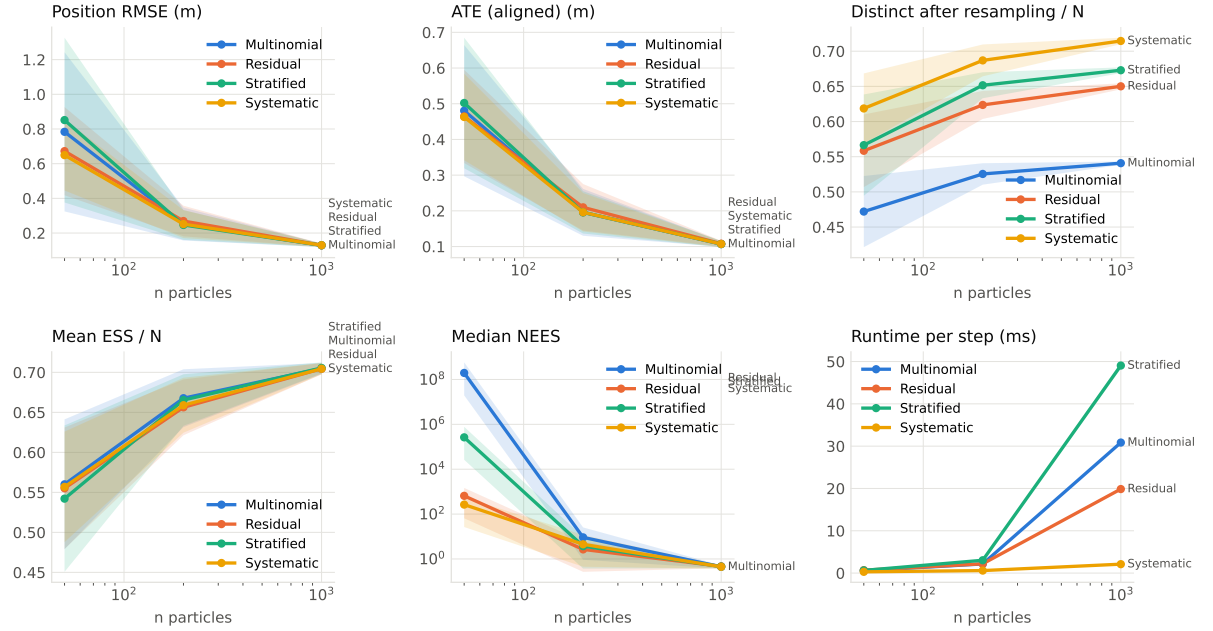


Figure 2: Resampling schemes in localization: metrics against the number of particles.

Table 2: Resampling schemes in FastSLAM 1.0, resampling at every step. Mean $\pm$ standard deviation over seeds.

Resampler	Particles	Seeds	Position RMSE (m)	ATE (aligned) (m)	Map error (aligned) (m)	Distinct after resampling / N	Mean ESS / N	Runtime per step (ms)
Multinomial	10	20	0.96 ± 0.55	0.324 ± 0.064	0.139 ± 0.04	0.624 ± 0.0043	0.893 ± 0.0051	1.5 ± 0.012
Multinomial	50	20	0.975 ± 0.47	0.223 ± 0.037	0.0965 ± 0.024	0.61 ± 0.002	0.893 ± 0.0025	2.71 ± 0.01
Residual	10	20	1.02 ± 0.84	0.277 ± 0.048	0.13 ± 0.037	0.785 ± 0.0024	0.892 ± 0.0041	1.57 ± 0.018
Residual	50	20	0.759 ± 0.41	0.209 ± 0.055	0.0832 ± 0.016	0.778 ± 0.0025	0.892 ± 0.0028	2.89 ± 0.02
Stratified	10	20	1.06 ± 0.71	0.247 ± 0.058	0.114 ± 0.037	0.801 ± 0.0042	0.893 ± 0.0043	1.51 ± 0.0073
Stratified	50	20	0.58 ± 0.31	0.205 ± 0.039	0.069 ± 0.02	0.787 ± 0.0016	0.892 ± 0.0021	2.8 ± 0.0088
Systematic	10	20	0.973 ± 0.61	0.233 ± 0.044	0.105 ± 0.032	0.86 ± 0.0038	0.895 ± 0.0037	1.46 ± 0.023
Systematic	50	20	0.879 ± 0.54	0.242 ± 0.1	0.0754 ± 0.016	0.858 ± 0.0019	0.892 ± 0.0024	2.35 ± 0.17

Does the resampling scheme matter? (FastSLAM 1.0)

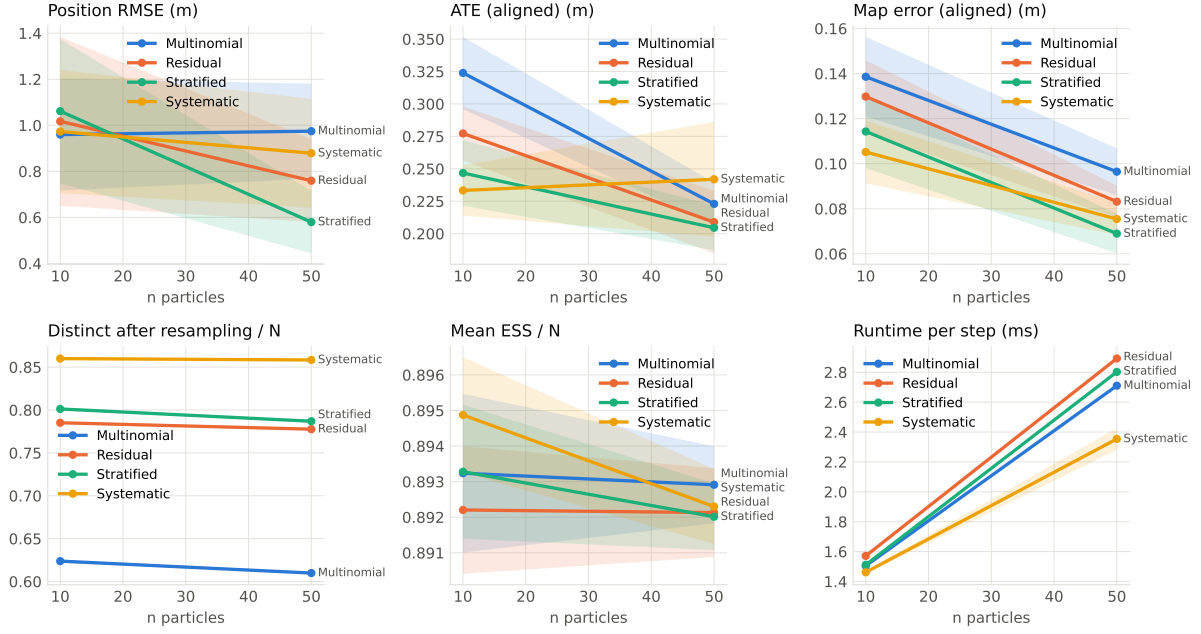


Figure 3: Resampling schemes in FastSLAM 1.0: metrics against the number of particles.

particles, the motion-model proposal and systematic resampling. The adaptive rules resample when $ESS < \tau N$ with $\tau = 0.2, 0.5$ or 0.8 , or when $1/\max_i w^{(i)} < \tau N$ with $\tau = 0.1, 0.2$ or 0.5 . The expectation was that never resampling loses track, resampling at every step wastes diversity, and an ESS threshold around 0.5 does best.

Results. Never resampling fails (Table 3): the effective sample size falls to about 1% of the particles and the position error, 1.39 m, is three times that of any rule that resamples. The median NEES of 1.6×10^9 shows that its estimate is confidently wrong. All seven rules that resample reach the same accuracy, 0.45 to 0.49 m, with differences far inside the seed-to-seed spread, and the same median NEES of 43 to 51. They differ in how often they resample: the thresholds of 0.2 resample at about a third of the steps (34% for the ESS rule, 35% for the largest-weight rule), the middle thresholds at about half (54% and 48%), and the highest at 75–80%. Resampling less often leaves fewer distinct particles after each resampling step, 25% at the lowest thresholds against 64% when resampling at every step, because the weights are more degenerate when resampling happens. With this proposal and world, the threshold therefore changes the cost and the diversity per resampling step, but not the accuracy; the ESS rule and the largest-weight rule behave almost identically at comparable thresholds.

Table 3: Rules for when to resample, localization with 100 particles. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Position RMSE (m)	Resampling rate	Distinct after resampling / N	Mean ESS / N	Median NEES
Every step	20	0.485 ± 0.37	1 ± 0	0.642 ± 0.093	0.591 ± 0.14	$46.1 \pm 1.4e+02$
Never	20	1.39 ± 0.44	0 ± 0	1 ± 0	0.0106 ± 0.00049	$1.6e+09 \pm 1.1e+09$
ESS threshold (0.2)	20	0.474 ± 0.34	0.342 ± 0.13	0.254 ± 0.017	0.351 ± 0.064	$50.5 \pm 1.5e+02$
ESS threshold (0.5)	20	0.458 ± 0.34	0.535 ± 0.13	0.438 ± 0.046	0.474 ± 0.089	$45.9 \pm 1.4e+02$
ESS threshold (0.8)	20	0.466 ± 0.36	0.8 ± 0.075	0.582 ± 0.089	0.565 ± 0.13	$45.6 \pm 1.4e+02$
Max weight (0.1)	20	0.477 ± 0.34	0.353 ± 0.15	0.261 ± 0.018	0.352 ± 0.059	$48.2 \pm 1.4e+02$
Max weight (0.2)	20	0.452 ± 0.32	0.483 ± 0.13	0.402 ± 0.04	0.444 ± 0.077	$43.3 \pm 1.4e+02$
Max weight (0.5)	20	0.483 ± 0.37	0.747 ± 0.087	0.556 ± 0.08	0.547 ± 0.12	$46 \pm 1.4e+02$

When should the filter resample?

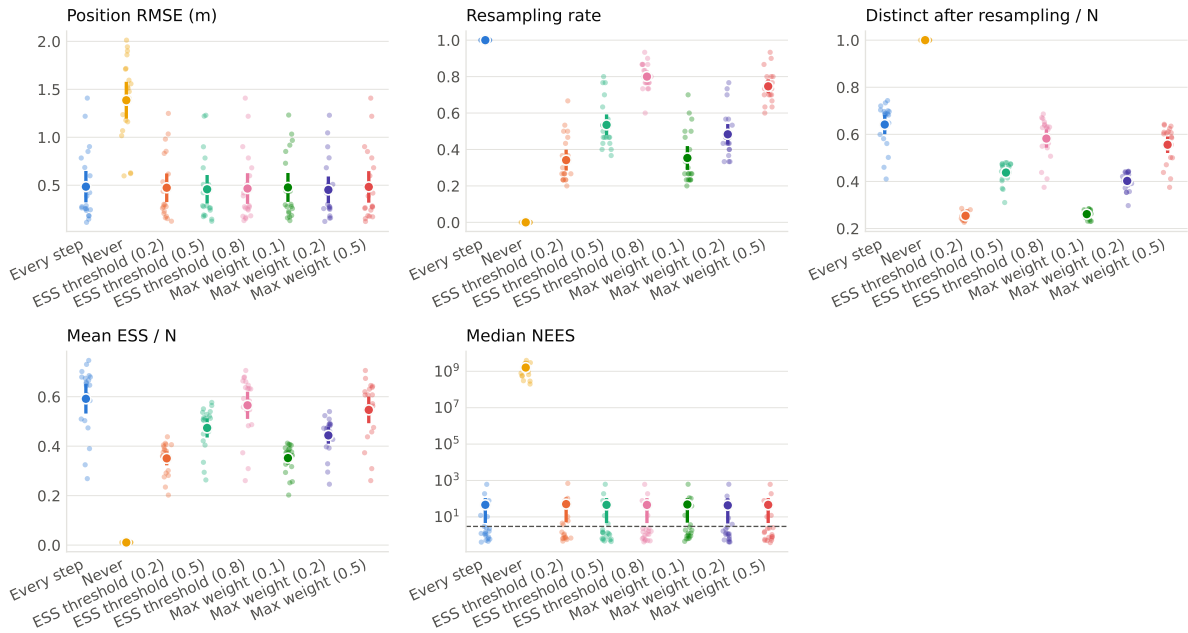


Figure 4: When to resample: metrics per rule.

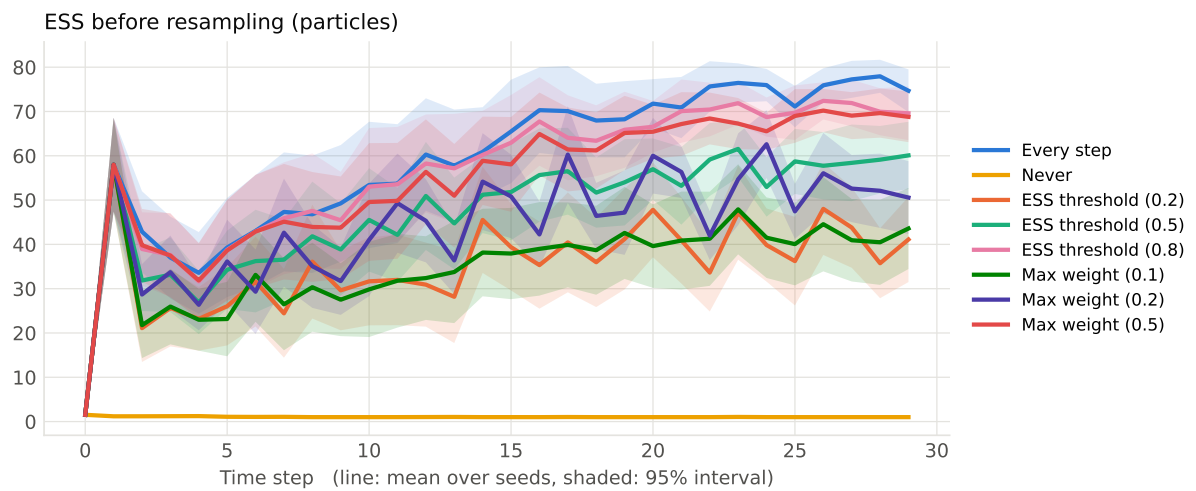


Figure 5: When to resample: effective sample size over time.

5.3 Proposal distribution

Question and setup. Do proposals that use the current measurement beat the motion model, and at which particle counts? In localization the motion model, the auxiliary particle filter and the extended Kalman particle filter run with 50, 100, 300 and 1000 particles, systematic resampling and an ESS threshold of 0.5. In SLAM, FastSLAM 1.0 and 2.0 run with 5, 10, 25 and 100 particles, systematic resampling and the upstream rule, resampling when $ESS < N/1.5$. The expectation was that measurement-informed proposals reach a given error with fewer particles at a higher cost per particle.

Localization. The auxiliary particle filter keeps the largest effective sample size at every particle count, 67–83% against 44–55% for the motion model (Table 4), and from 100 particles on it is the most accurate: 0.26 m against 0.46 m for the motion model at 100 particles, and 0.12 against 0.13 m at 1000. Its runtime per step is about twice that of the motion model with few particles and similar with 1000 particles. The extended Kalman particle filter keeps only about 20% effective particles at every particle count. Its error falls with more particles, from 0.59 m at 50 to 0.25 m at 1000, and with 50 particles it is as accurate as the other two, but with 1000 particles it is clearly worse than both (0.25 against 0.12 and 0.13 m) and six times slower. In this setting its linearized proposal does not make the weights more even, which is the point of a measurement-informed proposal (Section 6).

SLAM. FastSLAM 2.0 keeps a clearly larger effective sample size than FastSLAM 1.0 at every particle count, 79–81% against 75–76% (Table 5). Its advantage in accuracy is largest with few particles. With 5 particles it gives a clearly lower trajectory error (ATE 0.26 against 0.34 m) and a clearly better map (0.10 against 0.16 m); with 10 particles the trajectory error is still clearly lower (0.22 against 0.26 m). With 25 particles FastSLAM 1.0 has the clearly lower trajectory error (0.21 against 0.28 m), and with 100 particles the two are within the spread. FastSLAM 2.0 costs 43–55% more time per step. With 25 particles FastSLAM 1.0 builds a map as good as FastSLAM 2.0 with 5 to 10 particles, at a similar runtime.

Table 4: Proposal distributions in localization. Mean $\pm$ standard deviation over seeds.

Proposal	Particles	Seeds	Position RMSE (m)	ATE (aligned) (m)	Mean ESS / N	Median NEES	Runtime per step (ms)
Motion model	50	20	0.682 $\pm$ 0.49	0.484 $\pm$ 0.29	0.44 $\pm$ 0.11	304 $\pm$ 6.7e+02	0.256 $\pm$ 0.027
Motion model	100	20	0.458 $\pm$ 0.34	0.332 $\pm$ 0.21	0.474 $\pm$ 0.089	45.9 $\pm$ 1.4e+02	0.318 $\pm$ 0.028
Motion model	300	20	0.296 $\pm$ 0.39	0.219 $\pm$ 0.22	0.5 $\pm$ 0.1	38 $\pm$ 1.6e+02	0.588 $\pm$ 0.062
Motion model	1000	20	0.131 $\pm$ 0.027	0.107 $\pm$ 0.021	0.546 $\pm$ 0.017	0.461 $\pm$ 0.16	2.79 $\pm$ 0.72
Auxiliary PF	50	20	0.624 $\pm$ 0.58	0.426 $\pm$ 0.29	0.673 $\pm$ 0.21	515 $\pm$ 1.5e+03	0.45 $\pm$ 0.041
Auxiliary PF	100	20	0.264 $\pm$ 0.12	0.215 $\pm$ 0.093	0.786 $\pm$ 0.053	0.919 $\pm$ 0.88	0.56 $\pm$ 0.023
Auxiliary PF	300	20	0.21 $\pm$ 0.19	0.17 $\pm$ 0.14	0.799 $\pm$ 0.084	2.78 $\pm$ 9.5	1.02 $\pm$ 0.054
Auxiliary PF	1000	20	0.119 $\pm$ 0.017	0.0991 $\pm$ 0.014	0.833 $\pm$ 0.011	0.411 $\pm$ 0.15	2.48 $\pm$ 0.031
Extended Kalman PF	50	20	0.592 $\pm$ 0.46	0.322 $\pm$ 0.17	0.182 $\pm$ 0.079	1.76e+04 $\pm$ 7.1e+04	1.83 $\pm$ 0.024
Extended Kalman PF	100	20	0.47 $\pm$ 0.5	0.24 $\pm$ 0.13	0.192 $\pm$ 0.083	522 $\pm$ 1.3e+03	2.63 $\pm$ 0.046
Extended Kalman PF	300	20	0.356 $\pm$ 0.29	0.236 $\pm$ 0.13	0.202 $\pm$ 0.057	42.3 $\pm$ 1.7e+02	5.79 $\pm$ 0.14
Extended Kalman PF	1000	20	0.253 $\pm$ 0.14	0.181 $\pm$ 0.083	0.218 $\pm$ 0.026	0.667 $\pm$ 0.65	15.2 $\pm$ 0.87

5.4 Number of particles

Question and setup. How do error, particle diversity and runtime change with the number of particles, and where do more particles stop helping? The localization filter uses the motion-model proposal, systematic resampling and an ESS threshold of 0.5, with 10 to 2500 particles.

Results. The error falls steeply up to a few hundred particles and then levels off (Table 6): from 1.50 m with 10 particles to 0.46 m with 100, 0.21 m with 250 and 0.16 m with 500, and then only to 0.13 m with 1000 and 0.12 m with 2500. The spread over seeds shrinks in the same way,

Which proposal distribution? (localization)

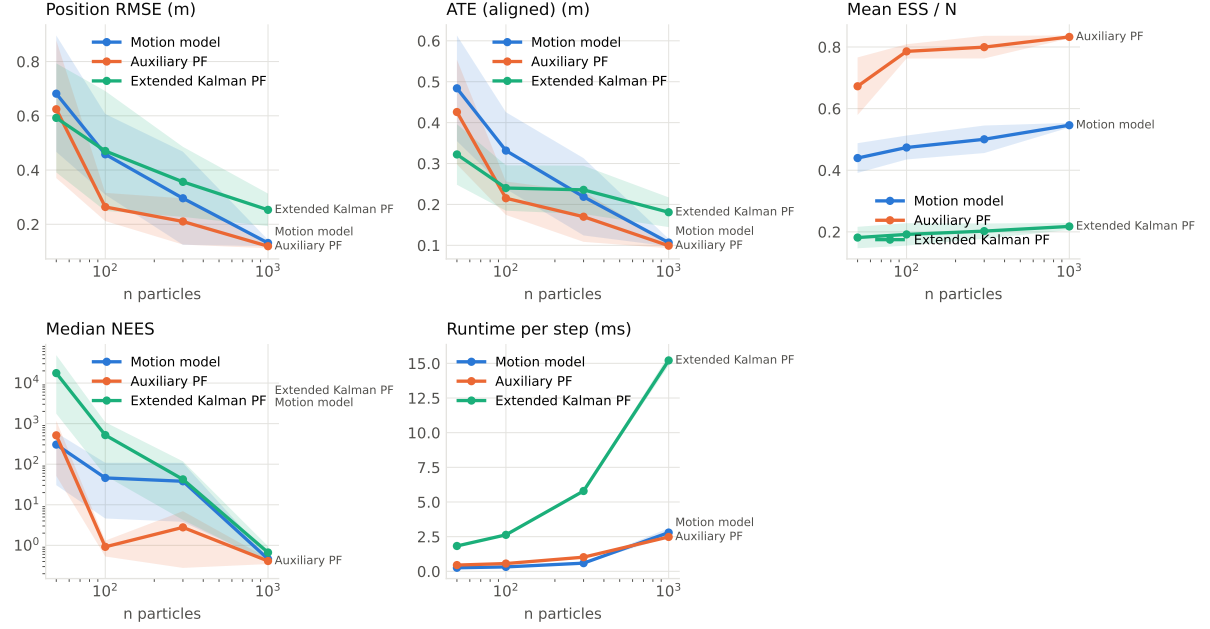


Figure 6: Proposal distributions in localization: metrics against the number of particles.

Table 5: FastSLAM 1.0 and 2.0. Mean $\pm$ standard deviation over seeds.

Proposal	Particles	Seeds	Position RMSE (m)	ATE (aligned) (m)	Map error (aligned) (m)	Mean ESS / N	Runtime per step (ms)
FastSLAM 1.0	5	20	1.12 ± 0.56	0.335 ± 0.052	0.16 ± 0.047	0.764 ± 0.0061	1.28 ± 0.0076
FastSLAM 1.0	10	20	0.958 ± 0.61	0.255 ± 0.033	0.111 ± 0.034	0.753 ± 0.0043	1.39 ± 0.0064
FastSLAM 1.0	25	20	0.673 ± 0.4	0.212 ± 0.05	0.0946 ± 0.025	0.75 ± 0.0023	1.86 ± 0.013
FastSLAM 1.0	100	20	0.777 ± 0.35	0.247 ± 0.094	0.0565 ± 0.016	0.748 ± 0.0022	3.19 ± 0.018
FastSLAM 2.0	5	20	0.773 ± 0.43	0.257 ± 0.037	0.102 ± 0.029	0.813 ± 0.0042	1.98 ± 0.013
FastSLAM 2.0	10	20	0.774 ± 0.47	0.215 ± 0.046	0.0909 ± 0.026	0.802 ± 0.0038	2.15 ± 0.012
FastSLAM 2.0	25	20	0.748 ± 0.47	0.275 ± 0.089	0.078 ± 0.015	0.795 ± 0.0025	2.66 ± 0.028
FastSLAM 2.0	100	20	0.737 ± 0.33	0.228 ± 0.067	0.0479 ± 0.012	0.793 ± 0.0024	4.79 ± 0.27

Which proposal distribution? (FastSLAM 1.0 vs 2.0)

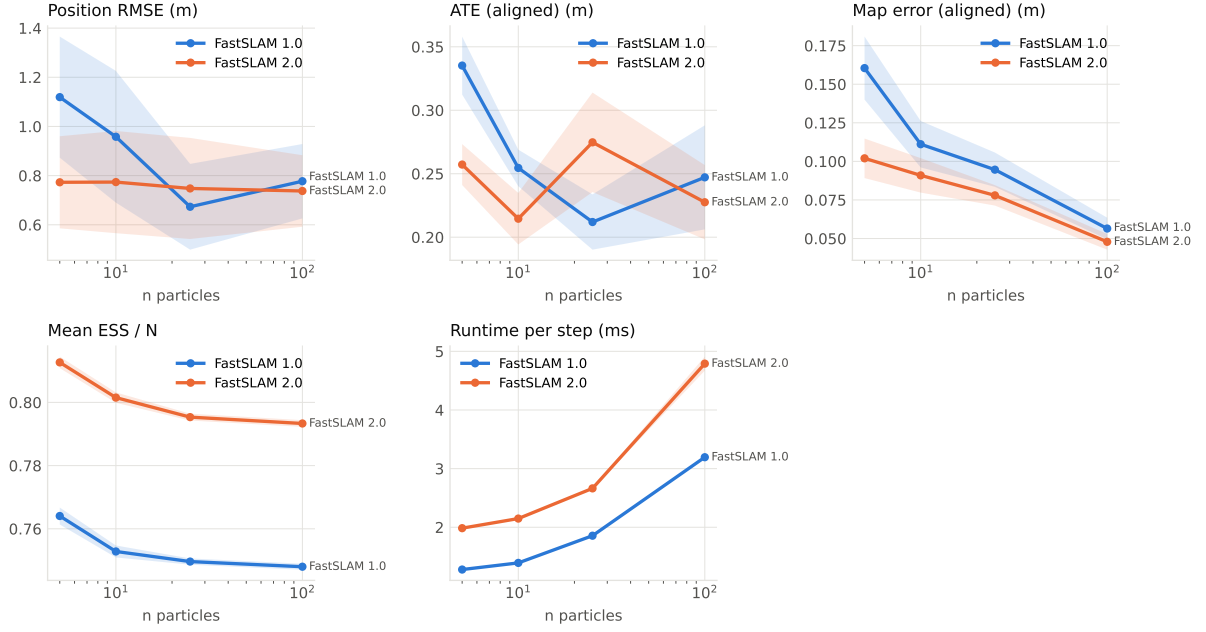


Figure 7: FastSLAM 1.0 and 2.0: metrics against the number of particles.

from 1.1 m to 0.02 m: with few particles some seeds do not find the true pose within the 30 steps after the uniform start, and with many particles all seeds do. The median NEES tells the same story. It is above 10^6 with 10 and 25 particles, where the particle cloud has collapsed onto a wrong pose, and falls below 3 from 250 particles on. Values below 3 mean the particle spread is somewhat larger than the actual error, which fits the filter’s assumed noise being larger than the true noise. The runtime per step grows from 0.18 ms to 3.7 ms. For this world, about 250 to 500 particles give most of the achievable accuracy at a small fraction of the cost of 2500.

Table 6: Number of particles, localization with the motion-model proposal. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Position RMSE (m)	ATE (aligned) (m)	Mean ESS / N	Median NEES	Runtime per step (ms)	Total runtime (s)
10	20	1.5 ± 1.1	0.779 ± 0.6	0.409 ± 0.12	$5.08e+08 \pm 2.1e+09$	0.182 ± 0.017	0.00546 ± 0.00052
25	20	1.1 ± 0.91	0.695 ± 0.43	0.405 ± 0.14	$1.83e+06 \pm 8.2e+06$	0.222 ± 0.019	0.00666 ± 0.00057
50	20	0.682 ± 0.49	0.484 ± 0.29	0.44 ± 0.11	$304 \pm 6.7e+02$	0.254 ± 0.026	0.00763 ± 0.00079
100	20	0.458 ± 0.34	0.332 ± 0.21	0.474 ± 0.089	$45.9 \pm 1.4e+02$	0.321 ± 0.032	0.00963 ± 0.00097
250	20	0.208 ± 0.16	0.168 ± 0.11	0.535 ± 0.045	1.92 ± 6.2	0.499 ± 0.031	0.015 ± 0.00092
500	20	0.158 ± 0.083	0.132 ± 0.057	0.538 ± 0.026	0.562 ± 0.39	0.807 ± 0.035	0.0242 ± 0.0011
1000	20	0.131 ± 0.027	0.107 ± 0.021	0.546 ± 0.017	0.461 ± 0.16	2.75 ± 0.72	0.0824 ± 0.022
2500	20	0.115 ± 0.018	0.0981 ± 0.015	0.549 ± 0.01	0.362 ± 0.13	3.69 ± 0.77	0.111 ± 0.023

5.5 Learning the noise parameters

Question and setup. The experiments above take the filter’s noise settings as given. Can they be estimated from a recorded run instead? Each of the four components of θ is swept in turn while the other three are held at the values the world actually uses, and every value is scored by the log likelihood (5) of the recorded measurements; the highest score gives the maximum-likelihood estimate. The filter uses the motion-model proposal, systematic resampling, an ESS threshold of 0.5 and 2000 particles, and starts around the true initial pose. The world is unchanged, so the answers are known: 0.2 m and 0.05 rad for the measurements, 0.005 m and 0.002 rad for the motion.

How many particles are enough?

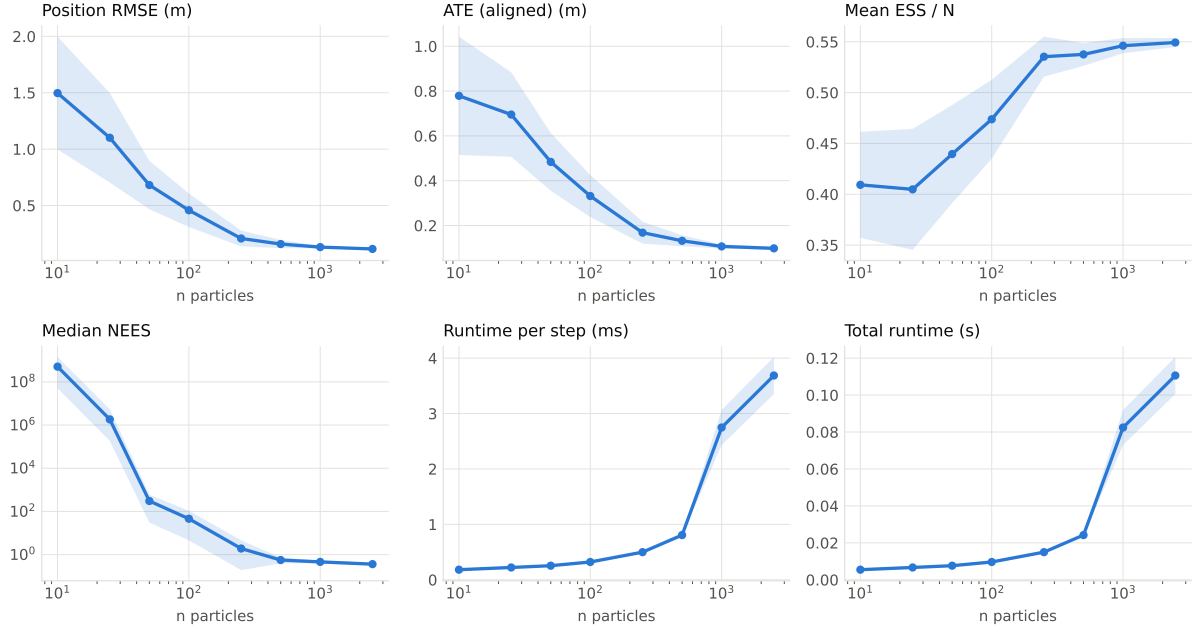


Figure 8: Accuracy and runtime against the number of particles.

The measurement parameters are recovered. The sweeps of the assumed range and bearing noise have a clear maximum at the true value (Tables 7 and 8). The best range noise is 0.19 m against a true 0.2 m, and the best bearing noise 0.047 rad against a true 0.05 rad, both within one step of the sweep. The shape on either side is what Section 2.4 predicts. Assuming too little noise makes the measurements that actually arrived look impossible and the log likelihood falls steeply, from 205 at 0.19 m to -252 at 0.08 m. Assuming too much noise makes every pose explain the measurements equally well and the score decays gently, to 98 at 0.8 m. The penalty is strongly one-sided: halving the assumed range noise costs about 450 in log likelihood, while quadrupling it costs about 100, the asymmetry that (8) predicts. A filter whose noise is set by hand is therefore much safer overstating it than understating it, which is what the upstream settings do.

On this world the most plausible setting is also close to the most accurate one. The position RMSE is lowest, 0.063 m, at 0.19 and 0.253 m of assumed range noise, the same place as the likelihood maximum, so learning the model and tuning for accuracy do not conflict here.

The sweep matches the closed form. The measured curve can be checked against (8), which gives the expected score of one residual, relative to its maximum, as $-\log(\sigma/\sigma^*) - (\sigma^{*2}/2\sigma^2 - 1/2)$. The range sweep has four landmarks over 30 steps, so 120 such residuals, and nothing is fitted. Relative to the peak, the prediction at 0.142, 0.253, 0.45 and 0.8 m is -18 , -6 , -49 and -110 , against measured -20 , -5 , -47 and -108 . The one place it parts company is the far left: at 0.08 m the prediction is -205 and the measurement -457 , because there the weights degenerate and the downward bias of the estimator, which (8) does not describe, dominates. The agreement is evidence that the sweep measures what it is meant to, and the disagreement marks where the particle approximation stops being trustworthy.

The motion parameters are not identifiable from 30 steps. The same sweeps over the assumed forward and turn noise are flat (Tables 9 and 10): the log likelihood stays at 205 to 206 while the assumed forward noise is varied by a factor of 30 and the assumed turn noise by a factor of 30. The true values, 0.005 m and 0.002 rad over 30 steps, are small next to the

measurement noise that the filter corrects with at every step, so the recorded measurements barely depend on them. The parameters are in the model, and the data say almost nothing about them.

This is a property of the data and not of the method. Repeating the turn sweep on a trajectory of 400 steps, with nothing else changed, turns the flat curve into one with a maximum at the true value (Fig. 10): 0.0025 rad against a true 0.002 rad, and an assumed noise of 0.0008 rad now costs about 8000 in log likelihood. Because the parameters are tied across time slices, every extra step is evidence about the same number, and by (7) the curvature of the likelihood grows with the number of steps, so enough steps resolve what 30 steps cannot. The one-sidedness is even stronger here: too small an assumed drift is rejected by thousands of nats, while ten times too much costs about 30.

Independent recordings agree exactly when the parameter is identifiable. Each seed is a separate recording of the robot, so the estimate can be formed from each seed on its own and the results compared. Where the sweep has a maximum the recordings agree: all 20 recordings pick 0.047 rad for the bearing noise, and 17 of 20 pick 0.19 m for the range noise with the other three picking the next value up. Where the sweep is flat they do not: the 20 estimates of the turn noise from 30-step runs are spread over the whole sweep, from 0.0008 to 0.025 rad, with no value chosen by more than a few recordings. On 400 steps they concentrate again, 9 of 20 on 0.0025 rad and all within 0.0008 to 0.0045 rad. Agreement between recordings is thus a usable test of whether a parameter is being estimated at all, and it also tests the assumption behind the tying: a noise level that changed between runs would show as a wide scatter even where the likelihood is sharply peaked.

Learning needs the filter to track. Repeating the range sweep from the uniform start used in Sections 5.1 to 5.4 removes the maximum entirely (Fig. 11). With the true, small noise the filter cannot find the robot within 30 steps from a uniform prior: the position RMSE is 2.3 m against 0.063 m from a known start, and the log likelihood is -1.3×10^4 against 205. The score then keeps improving as the assumed noise is widened, to -5.2×10^3 at 0.8 m, because a wider noise lets the particles spread enough to be useful, and the estimates from individual recordings scatter across the whole sweep, with only 2 of 20 agreeing. What the likelihood reports in this regime is which setting lets this filter work, not which setting generated the data.

How much inference learning needs. Repeating the range sweep with 25 to 2000 particles shows the transition (Fig. 12). At the true value the estimated log likelihood rises from 28 ± 316 with 25 particles to 184 ± 65 with 100, 203 ± 16 with 500 and 205 ± 13 with 2000: the downward bias and the spread of the estimator both shrink as the particle approximation improves, as Section 2.4 anticipates. The estimate follows. With 25 particles no two recordings agree and the median estimate is 0.22 m; with 100 particles 12 of 20 recordings pick 0.19 m, with 500 particles 14 of 20 and with 2000 particles 17 of 20. The resampling scheme, in contrast, makes almost no difference (Table 11): with 200 particles all four schemes put the maximum at 0.19 m and 14 or 15 of 20 recordings agree, matching Section 5.1, where the schemes differed in particle diversity rather than in accuracy.

6 Discussion

Which choices mattered. The number of particles and the decision to resample at all had by far the largest effects on accuracy. Going from 10 to 500 particles reduced the localization error almost tenfold, and a filter that never resamples was three times worse than any filter that does. The choices within resampling mattered much less. In localization the four schemes and seven rules were equally accurate. The schemes differed in how many distinct particles survive, with

Can the measurement noise be learned from the data? (range)

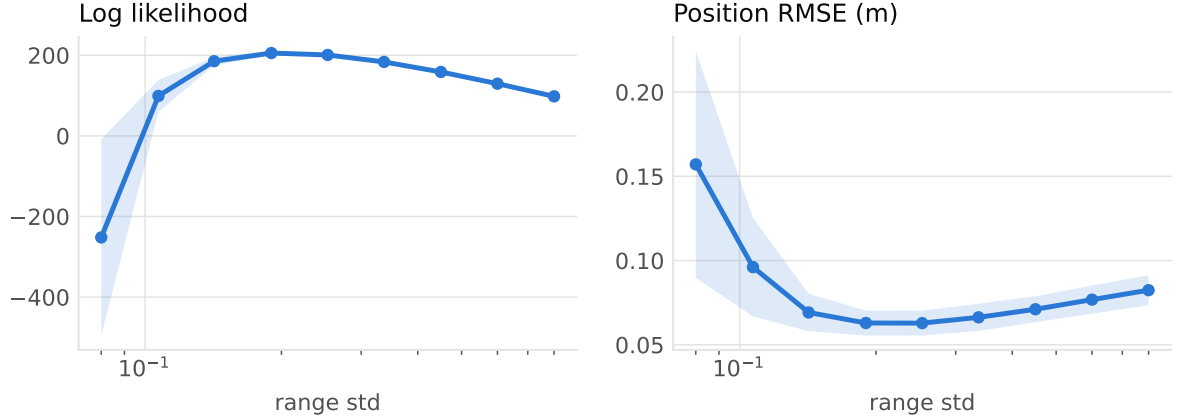


Figure 9: Sweeping the assumed range noise: the log likelihood peaks at the value the world uses, 0.2m, and the position error is lowest at the same place.

Table 7: Sweeping the assumed range noise; the world uses 0.2m. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Log likelihood	Position RMSE (m)
0.08	20	$-252 \pm 5.6e+02$	0.157 ± 0.15
0.107	20	99.1 ± 90	0.0961 ± 0.067
0.142	20	185 ± 24	0.0692 ± 0.025
0.19	20	205 ± 13	0.0629 ± 0.017
0.253	20	201 ± 8.7	0.0629 ± 0.017
0.337	20	183 ± 6.5	0.0663 ± 0.018
0.45	20	158 ± 5.7	0.0711 ± 0.018
0.6	20	129 ± 5.6	0.0768 ± 0.019
0.8	20	98 ± 5.7	0.0824 ± 0.02

Table 8: Sweeping the assumed bearing noise; the world uses 0.05rad. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Log likelihood	Position RMSE (m)
0.02	20	$-45.8 \pm 1.2e+02$	0.126 ± 0.075
0.0267	20	130 ± 30	0.0836 ± 0.031
0.0356	20	190 ± 15	0.0682 ± 0.019
0.0474	20	206 ± 12	0.0626 ± 0.017
0.0632	20	200 ± 11	0.0624 ± 0.017
0.0843	20	181 ± 10	0.0653 ± 0.017
0.1125	20	156 ± 10	0.0698 ± 0.019
0.15	20	127 ± 9.9	0.0736 ± 0.02
0.2	20	95.1 ± 9.8	0.0779 ± 0.022

Table 9: Sweeping the assumed forward motion noise; the world uses 0.005 m. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Log likelihood	Position RMSE (m)
0.002	20	206 $\pm$ 12	0.0636 $\pm$ 0.018
0.00356	20	206 $\pm$ 12	0.0638 $\pm$ 0.018
0.00632	20	206 $\pm$ 12	0.063 $\pm$ 0.017
0.01125	20	206 $\pm$ 12	0.0626 $\pm$ 0.016
0.02	20	206 $\pm$ 12	0.0645 $\pm$ 0.016
0.0356	20	205 $\pm$ 12	0.0679 $\pm$ 0.014
0.0632	20	202 $\pm$ 12	0.0771 $\pm$ 0.018

Table 10: Sweeping the assumed turn noise over 30 steps; the world uses 0.002 rad. Mean $\pm$ standard deviation over seeds.

Variant	Seeds	Log likelihood	Position RMSE (m)
0.0008	20	205 $\pm$ 12	0.0637 $\pm$ 0.018
0.00142	20	205 $\pm$ 12	0.0633 $\pm$ 0.017
0.00253	20	206 $\pm$ 12	0.0637 $\pm$ 0.018
0.0045	20	206 $\pm$ 12	0.0629 $\pm$ 0.017
0.008	20	206 $\pm$ 12	0.0632 $\pm$ 0.017
0.0142	20	206 $\pm$ 12	0.0632 $\pm$ 0.015
0.0253	20	205 $\pm$ 12	0.065 $\pm$ 0.014

Does more data make the motion noise identifiable? (turn, 400 steps)

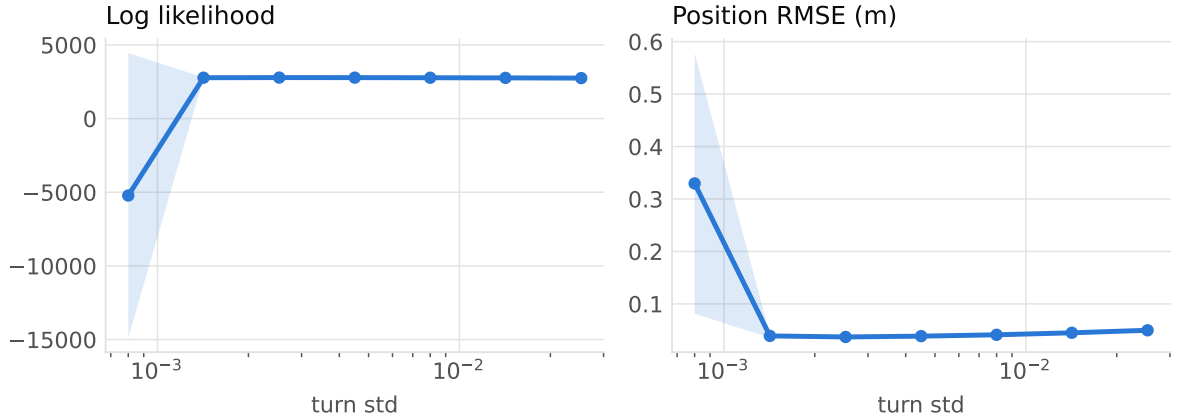


Figure 10: The same turn sweep over 400 steps: enough steps identify a value that 30 steps cannot.

Table 11: The range noise picked out under each resampling scheme, with 200 particles.

Resampler	Seeds	Best assumed range noise (median)	Range over seeds	Seeds at the median
Multinomial	20	0.19	0.19 to 0.45	15 of 20
Residual	20	0.19	0.19 to 0.337	14 of 20
Stratified	20	0.19	0.19 to 0.253	15 of 20
Systematic	20	0.19	0.19 to 0.337	14 of 20

Does learning depend on the filter tracking at all?

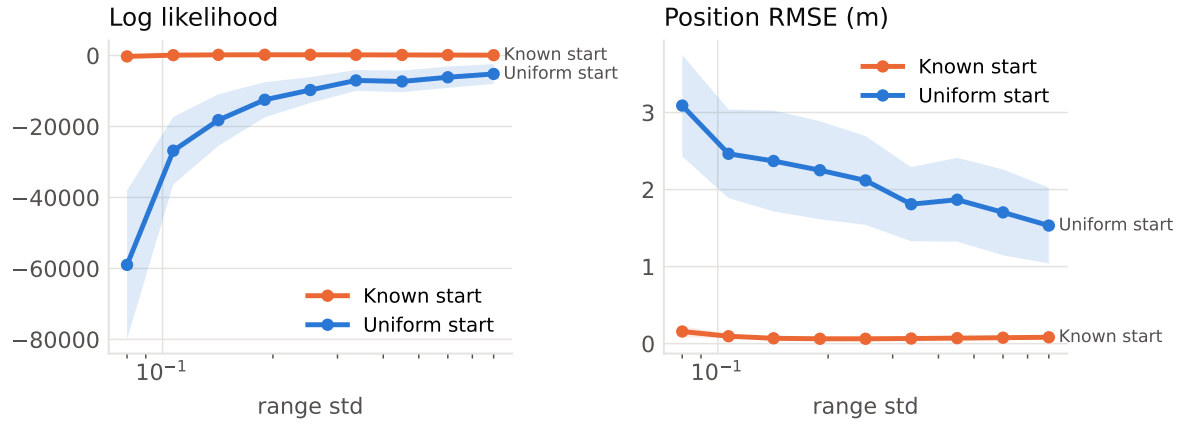


Figure 11: The range sweep from a known start and from the uniform start of Sections 5.1 to 5.4. Note the scale: from the uniform start the estimate is four orders of magnitude lower and has no maximum.

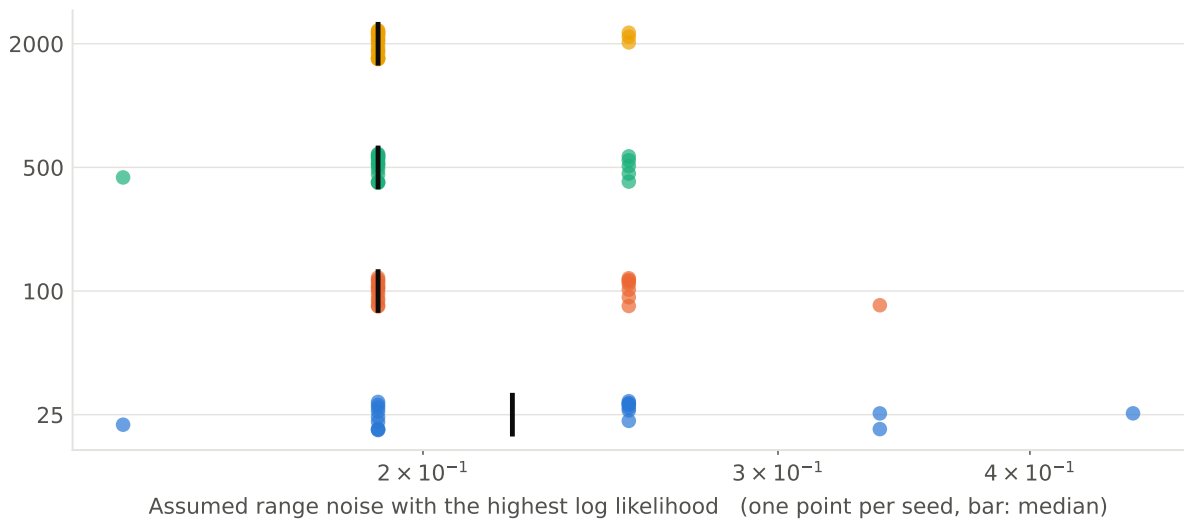


Figure 12: The range noise that each recording picks out on its own, against the number of particles.

the known order from multinomial to systematic resampling [2, 8], and the rules differed in how often they resample. These differences became visible in accuracy only in FastSLAM with 10 particles, where losing a particle means losing a whole map hypothesis. The proposal mattered in between: a good measurement-informed proposal, the auxiliary particle filter in localization and FastSLAM 2.0 in SLAM, helped most when particles are scarce, and its advantage shrank or vanished with more particles.

Trade-offs. Measurement-informed proposals cost more per particle, so the fair comparison is at equal runtime rather than at equal particle count. At about 0.6 ms per step, the auxiliary particle filter with 100 particles reached 0.26 m and the motion model with 300 particles 0.30 m, a difference within the seed-to-seed spread; the auxiliary filter’s clear advantage is at equal particle count. FastSLAM 2.0 with 5 to 10 particles and FastSLAM 1.0 with 25 particles built maps of similar quality in similar time. Adaptive resampling brings no gain in accuracy in these worlds, but it saves resampling steps at no loss. Resampling less often leaves each resampling step with more degenerate weights, and therefore with fewer distinct particles afterwards, so the threshold trades cost against impoverishment per step.

The extended Kalman particle filter. The extended Kalman particle filter gave the lowest effective sample size of all proposals and, with many particles, a higher error than the motion model. A measurement-informed proposal is meant to approximate $p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{z}_t, \mathbf{u}_t)$ so that the weights (9) become more even. A likely reason why it does not here lies in the form used: each particle’s Kalman covariance is carried from step to step and grows with the accumulated uncertainty of the filter, while the weights compare the sample with a single motion step. When the proposal is much wider or differently shaped than the posterior of one step, most samples receive small weights. Linearizing the measurement model about each particle’s own pose, far from the true pose after the uniform start, adds to this. A proposal built from the motion noise of one step alone, the linearized form of the optimal proposal [3], is the natural alternative and a candidate for further experiments.

The graphical-model view. The experiments can be read as a study of inference design choices on a dynamic Bayesian network. The proposal decides how much of the evidence $\mathbf{z}_t$ enters the sampling step and how much is left to the weights: the motion-model proposal uses only the parent $\mathbf{x}_{t-1}$ of $\mathbf{x}_t$, while the measurement-informed proposals also condition on its child $\mathbf{z}_t$. Rao-Blackwellization decides which variables are sampled at all. In FastSLAM only the trajectory is sampled and the map is handled exactly given it, which is why a handful of particles already produces a usable map. Resampling has no counterpart in exact inference. It is the price of representing a belief by a finite sample, and its variants only differ in how much extra noise they add while paying it.

What limits learning. Estimating the noise parameters from a run uses the same weights as the filtering itself, and the experiments show it inherits the filter’s weaknesses twice over. A filter that has not found the robot gives a likelihood that rewards whichever setting lets it work rather than the setting that generated the data, and a filter with too few particles gives an estimate so noisy and so biased that no two recordings agree. Inference quality is therefore a precondition for learning, not merely something learning improves. The data set a second limit that no amount of computation removes: the two motion parameters are perfectly well defined and completely undetermined by 30 steps of this world, and become determined only once the trajectory is long enough for their effect to show against the measurement noise. Agreement between independent recordings separates the two cases cheaply, and it is also the test of the tying assumption that lets one parameter stand for every time slice. The one-sided shape of the

likelihood explains the upstream settings: understating noise is punished far more heavily than overstating it, so noise values chosen by hand are rationally kept large.

Limitations. The worlds are small and simulated: a 30-step localization run in a 10×10 m world with four landmarks, and one SLAM route with eight landmarks and known correspondences. Real sensors, unknown data association and larger maps would make degeneracy and proposal quality more important. The filter noise settings in Sections 5.1 to 5.4 are those of the upstream code, and they are larger than the true noise; filters tuned closer to the true noise would have more peaked likelihoods and more degenerate weights, which could change the ranking of the proposals and rules. The learning experiments vary those settings but only one at a time, with the others held at the true values and from a known start, so they do not say what the maximum-likelihood setting would be if all four were estimated together, and the parameters were estimated by a sweep rather than by expectation maximization. Each experiment varies one choice with the others fixed, so interactions, for example between the resampling scheme and the rule for when to resample, are not measured. The rules for when to resample were compared only in localization, the resampling schemes in SLAM only with FastSLAM 1.0, and the auxiliary and extended Kalman proposals only in localization. Runtimes were measured with 16 runs in parallel and depend on the reused implementations, as for the resampling schemes. MCMC moves after resampling [6], which address impoverishment directly, were not evaluated.

7 Conclusion

This report treated robot localization and SLAM as dynamic Bayesian networks and the particle filter as approximate inference on them: the exact filtering recursion with importance sampling in place of the integrals, and for SLAM a Rao-Blackwellized split into a sampled trajectory and per-landmark Kalman filters, justified by the conditional independence of the landmarks given the trajectory. On this basis it compared the sampling choices of the filter in controlled experiments with 20 simulated worlds per configuration.

The number of particles and the use of resampling decided most of the accuracy. The four resampling schemes were equally accurate in localization, with systematic resampling keeping the most distinct particles and, in FastSLAM with few particles, giving a better trajectory and map than multinomial resampling. Resampling only when the effective sample size or the largest weight signals degeneracy was as accurate as resampling at every step while resampling at a third to a half of the steps with low thresholds. Proposals that use the current measurement kept more particles effective and helped most when particles are few: the auxiliary particle filter was the most accurate localization filter from 100 particles on, and FastSLAM 2.0 built better maps than FastSLAM 1.0 with 5 and 10 particles. The extended Kalman particle filter, in the form tested here, did not improve on the motion model. For the localization world, a few hundred particles gave most of the achievable accuracy.

Turning the question around, the noise parameters of the two conditional distributions could be estimated from a recorded run by the same weights the filter already forms. The measurement parameters were recovered to within one step of the sweep, 0.19 m against a true 0.2 m and 0.047 rad against a true 0.05 rad, and independent recordings agreed on them. The motion parameters were not determined by 30 steps of this world at all, and became determined on 400 steps, so identifiability is a property of the data rather than of the method. Learning also proved to rest on the inference: from a uniform start, or with too few particles, the likelihood does not point at the values that generated the data.

The study also showed that reused filter code has to be checked before it is compared: the original FastSLAM 2.0 code did not sample from its proposal, and several other errors would have biased the results. Natural extensions are MCMC moves after resampling, the linearized optimal proposal for the extended Kalman particle filter, the resampling rules in SLAM, and

real data with unknown correspondences.

A Upstream Code and Corrections

Sources. The localization filters, the resampling schemes and the rules for when to resample come from the particle filter tutorial of Elfring et al. [5] (<https://github.com/jelfring/particle-filter-tutorial>, commit e6014b7, MIT licence). FastSLAM 1.0 and 2.0 come from PythonRobotics [13] (<https://github.com/AtsushiSakai/PythonRobotics>, SLAM/FastSLAM1 and SLAM/FastSLAM2, commit 1fe4fb9, MIT licence). The upstream code is downloaded at these commits and never modified. The resampling schemes and rules are called unchanged; the models, proposals and FastSLAM updates are copied, vectorized over particles, and changed to work with log weights.

Checking. The corrections to the models and proposals can be switched off to restore the upstream behaviour. With them switched off, the copies were compared with the upstream functions on the same inputs and random numbers. The localization likelihood and the noise-free motion model match exactly, and one full extended Kalman particle filter update matches in poses, covariances and weights when both use the Gaussian mean instead of a random draw. The FastSLAM prediction matches exactly with the same seed, and the FastSLAM 1.0 and 2.0 observation updates match in weights, landmark means and covariances over two steps. These checks are part of the test suite.

Corrections in the localization code.

1. The likelihood compares the expected and measured bearing without wrapping the difference, so a small error across $\pm\pi$ looks like an error of almost 2π and the particle gets almost no weight. The difference is wrapped to $[-\pi, \pi)$.
2. The likelihood is the exponent of a Gaussian without its normalizing constant. The constant is the same for every particle, so it cancels when the weights are normalized and no result of Sections 5.1 to 5.4 depends on it, but it is exactly the term that penalizes a wider assumed noise. Without it the marginal likelihood (5) grows without bound as the assumed measurement noise grows, and no parameter can be estimated from it. The full log density is used.
3. Extended Kalman particle filter: the covariance prediction multiplies element by element instead of computing the matrix product $FPF^\top$; the noise covariances are built from standard deviations instead of variances; the bearing innovation is not wrapped; the prior is centred on the updated state instead of the prediction, because the predicted state and the updated state are the same list object; and the bearing Jacobian divides by zero when the particle and the landmark have the same x coordinate. All five are corrected.
4. Extended Kalman particle filter: positions wrap around at the edges of the world, but the prior and proposal densities use raw position differences. A pose sampled just across an edge then appears a world-width away from its prediction and can receive an arbitrarily large weight; from a uniform start this makes all particles collapse onto one wrong pose more often the more particles there are. The differences are wrapped to the world. A test checks that the mean weight from a uniform start equals the mean likelihood under the prior, as importance sampling requires; without the correction the two differ by a factor of more than e^{10^4} .
5. The extended Kalman particle filter hard-codes multinomial resampling at every step, and the auxiliary particle filter hard-codes multinomial sampling in its first stage. Resampling

is left to the configured scheme and rule, and the auxiliary filter’s first-stage selection uses the configured scheme.

6. The systematic and stratified search loops can step past the last particle when rounding leaves the cumulative weight slightly below one. The last weight is raised by that shortfall before calling them.

Corrections in the FastSLAM code.

1. The FastSLAM 2.0 proposal moves each particle to the mean of its proposal Gaussian but never draws a sample from it, never corrects the weight for the proposal, and keeps the proposal covariance from one step to the next. This is closer to a Kalman update per particle than to FastSLAM 2.0, and comparing it with FastSLAM 1.0 would not compare a motion-model proposal with a measurement-informed one. The corrected proposal follows Montemerlo et al. [11]: it predicts the pose, updates the pose Gaussian with each observation of an already mapped landmark while weighting by the predictive likelihood, samples the pose, and then updates the landmark filters.
2. Resampling copies the list of particle objects but not the particles, so resampled particles share their state and their landmark arrays. Particles are stored as arrays and copied on resampling.
3. A landmark counts as new when its estimated x coordinate is within 0.01 of zero, so a landmark estimated near $x = 0$ is initialized again. A flag per particle and landmark records which landmarks have been seen.
4. The pose estimate averages headings as plain numbers, which fails near $\pm\pi$. A circular mean is used.
5. Resampling is hard-coded (low-variance resampling when $\text{ESS} < N/1.5$). The configured scheme and rule are used, with this rule as the default.

Effect. With the upstream code, the comparison of FastSLAM 1.0 and 2.0 in Section 5.3 would have compared two variants of a motion-model proposal. Without the normalizing constant the sweeps of Section 5.5 rise monotonically with the assumed measurement noise and have no maximum. Without the correction at the world’s edges, the extended Kalman particle filter becomes less accurate as particles are added. The results in Section 5 use the corrected code throughout.

References

- [1] M. Sanjeev Arulampalam et al. “A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking”. In: *IEEE Transactions on Signal Processing* 50.2 (2002), pp. 174–188. DOI: [10.1109/78.978374](https://doi.org/10.1109/78.978374).
- [2] Randal Douc, Olivier Cappé, and Eric Moulines. “Comparison of resampling schemes for particle filtering”. In: *Proceedings of the International Symposium on Image and Signal Processing and Analysis (ISPA)*. 2005, pp. 64–69. DOI: [10.1109/ISPA.2005.195385](https://doi.org/10.1109/ISPA.2005.195385).
- [3] Arnaud Doucet and Adam M. Johansen. “A tutorial on particle filtering and smoothing: Fifteen years later”. In: *The Oxford Handbook of Nonlinear Filtering*. Oxford University Press, 2011. URL: https://www.stats.ox.ac.uk/~doucet/doucet_johansen_tutorialPF2011.pdf.

- [4] Arnaud Doucet et al. “Rao-Blackwellised particle filtering for dynamic Bayesian networks”. In: *Proceedings of the Conference on Uncertainty in Artificial Intelligence (UAI)*. 2000. arXiv: [1301.3853](#).
- [5] Jos Elfring, Elena Torta, and René van de Molengraft. “Particle Filters: A Hands-On Tutorial”. In: *Sensors* 21.2 (2021), p. 438. DOI: [10.3390/s21020438](#).
- [6] Walter R. Gilks and Carlo Berzuini. “Following a moving target—Monte Carlo inference for dynamic Bayesian models”. In: *Journal of the Royal Statistical Society: Series B* 63.1 (2001), pp. 127–146. DOI: [10.1111/1467-9868.00280](#).
- [7] Neil J. Gordon, David J. Salmond, and Adrian F. M. Smith. “Novel approach to nonlinear/non-Gaussian Bayesian state estimation”. In: *IEE Proceedings F (Radar and Signal Processing)* 140.2 (1993), pp. 107–113. DOI: [10.1049/ip-f-2.1993.0015](#).
- [8] Jeroen D. Hol, Thomas B. Schön, and Fredrik Gustafsson. “On resampling algorithms for particle filters”. In: *IEEE Nonlinear Statistical Signal Processing Workshop*. 2006, pp. 79–82. DOI: [10.1109/NSSPW.2006.4378824](#).
- [9] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009. ISBN: 978-0-262-01319-2. URL: <https://mitpress.mit.edu/9780262013192/probabilistic-graphical-models/>.
- [10] Michael Montemerlo et al. “FastSLAM: A factored solution to the simultaneous localization and mapping problem”. In: *Proceedings of the AAAI National Conference on Artificial Intelligence*. 2002. URL: <https://cdn.aaai.org/AAAI/2002/AAAI02-089.pdf>.
- [11] Michael Montemerlo et al. “FastSLAM 2.0: An improved particle filtering algorithm for simultaneous localization and mapping that provably converges”. In: *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*. 2003. URL: <https://www.ijcai.org/Proceedings/03/Papers/165.pdf>.
- [12] Michael K. Pitt and Neil Shephard. “Filtering via simulation: Auxiliary particle filters”. In: *Journal of the American Statistical Association* 94.446 (1999), pp. 590–599. DOI: [10.1080/01621459.1999.10474153](#).
- [13] Atsushi Sakai et al. *PythonRobotics: a Python code collection of robotics algorithms*. 2018. arXiv: [1808.10703](#). URL: <https://github.com/AtsushiSakai/PythonRobotics>.
- [14] Sebastian Thrun, Wolfram Burgard, and Dieter Fox. *Probabilistic Robotics*. MIT Press, 2005. ISBN: 978-0-262-20162-9. URL: <https://mitpress.mit.edu/9780262201629/probabilistic-robotics/>.